JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ConcurrentHashMap.KeySetView<K,V>

java.lang.Object
  java.util.concurrent.ConcurrentHashMap.KeySetView<K,V>

Type Parameters:

K - the type of keys

V - the type of values in the backing map

All Implemented Interfaces:

Serializable, Iterable<K>, Collection<K>, Set<K>

Enclosing class:

ConcurrentHashMap<K,V>

```
public static final class ConcurrentHashMap.KeySetView<K,V>
extends Object
implements Set<K>, Serializable
```

A view of a ConcurrentHashMap as a Set of keys, in which additions may optionally be enabled by mapping to a common value. This class cannot be directly instantiated. See keySet(), keySet(V), newKeySet(), newKeySet(int).

Since:

1.8

See Also:

Serialized Form

Method Summary

All Methods	Instance Methods	
Modifier and Type	**Method**	**Description**
boolean	**add**(K e)	Adds the specified key to this set view by mapping the key to the default mapped value in the backing map, if defined.
boolean	**addAll**(Collection<? extends K> c)	Adds all of the elements in the specified collection to this set, as if by calling add(K) on each one.
final void	**clear**()	

		Removes all of the elements from this view, by removing all the mappings from the map backing this view.
boolean	**contains**(Object o)	Returns true if this collection contains the specified element.
final boolean	**containsAll**(Collection<?> c)	Returns true if this collection contains all of the elements in the specified collection.
boolean	**equals**(Object o)	Compares the specified object with this collection for equality.
ConcurrentHashMap<K,V>	**getMap**()	Returns the map backing this view.
V	**getMappedValue**()	Returns the default mapped value for additions, or null if additions are not supported.
int	**hashCode**()	Returns the hash code value for this collection.
final boolean	**isEmpty**()	Returns true if this collection contains no elements.
Iterator<K>	**iterator**()	Returns an iterator over the elements in this collection.
boolean	**remove**(Object o)	Removes the key from this map view, by removing the key (and its corresponding value) from the backing map.
boolean	**removeAll**(Collection<?> c)	Removes all of this collection's elements

		that are also contained in the specified collection (optional operation).
final boolean	**retainAll**(Collection<? > c)	Retains only the elements in this collection that are contained in the specified collection (optional operation).
final int	**size**()	Returns the number of elements in this collection.
final Object[]	**toArray**()	Returns an array containing all of the elements in this collection.
final <T> T[]	**toArray**(T[] a)	Returns an array containing all of the elements in this collection; the runtime type of the returned array is that of the specified array.
final String	**toString**()	Returns a string representation of this collection.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Methods declared in interface Collection

parallelStream, removeIf, stream, toArray

Methods declared in interface Iterable

`forEach`

Methods declared in interface Set

`clear`, `containsAll`, `isEmpty`, `removeAll`, `retainAll`, `size`, `spliterator`, `toArray`, `toArray`

Method Details

getMappedValue

`public V getMappedValue()`

Returns the default mapped value for additions, or `null` if additions are not supported.

Returns:

the default mapped value for additions, or `null` if not supported

contains

`public boolean contains(Object o)`

Returns `true` if this collection contains the specified element. More formally, returns `true` if and only if this collection contains at least one element `e` such that `Objects.equals(o, e)`.

Specified by:

`contains` in interface `Collection<K>`

Specified by:

`contains` in interface `Set<K>`

Parameters:

`o` - element whose presence in this collection is to be tested

Returns:

`true` if this collection contains the specified element

Throws:

`NullPointerException` - if the specified key is null

remove

`public boolean remove(Object o)`

Removes the key from this map view, by removing the key (and its corresponding value) from the backing map. This method does nothing if the key is not in the map.

Specified by:

`remove` in interface `Collection<K>`

Specified by:

`remove` in interface `Set<K>`

Parameters:

`o` - the key to be removed from the backing map

Returns:

`true` if the backing map contained the specified key

Throws:

`NullPointerException` - if the specified key is null

iterator

```
public Iterator<K> iterator()
```

Returns an iterator over the elements in this collection.

The returned iterator is *weakly consistent*.

Specified by:

`iterator` in interface `Collection<K>`

Specified by:

`iterator` in interface `Iterable<K>`

Specified by:

`iterator` in interface `Set<K>`

Returns:

an iterator over the keys of the backing map

add

```
public boolean add(K e)
```

Adds the specified key to this set view by mapping the key to the default mapped value in the backing map, if defined.

Specified by:

`add` in interface `Collection<K>`

Specified by:

`add` in interface `Set<K>`

Parameters:

e - key to be added

Returns:

`true` if this set changed as a result of the call

Throws:

`NullPointerException` - if the specified key is null

`UnsupportedOperationException` - if no default mapped value for additions was provided

addAll

```
public boolean addAll(Collection<? extends K> c)
```

Adds all of the elements in the specified collection to this set, as if by calling `add(K)` on each one.

Specified by:

`addAll` in interface `Collection<K>`

Specified by:

`addAll` in interface `Set<K>`

Parameters:

c - the elements to be inserted into this set

Returns:

`true` if this set changed as a result of the call

Throws:

`NullPointerException` - if the collection or any of its elements are `null`

`UnsupportedOperationException` - if no default mapped value for additions was provided

See Also:

`Collection.add(Object)`

hashCode

```
public int hashCode()
```

Description copied from interface: `Collection`

Returns the hash code value for this collection. While the `Collection` interface adds no stipulations to the general contract for the `Object.hashCode` method, programmers should take note that any class that overrides the `Object.equals` method must also override the `Object.hashCode` method in

order to satisfy the general contract for the `Object.hashCode` method. In particular, `c1.equals(c2)` implies that `c1.hashCode()==c2.hashCode()`.

Specified by:

`hashCode` in interface `Collection<K>`

Specified by:

`hashCode` in interface `Set<K>`

Overrides:

`hashCode` in class `Object`

Returns:

the hash code value for this collection

See Also:

`Object.hashCode()`, `Object.equals(Object)`

equals

```
public boolean equals(Object o)
```

Description copied from interface: `Collection`

Compares the specified object with this collection for equality.

While the `Collection` interface adds no stipulations to the general contract for the `Object.equals`, programmers who implement the `Collection` interface "directly" (in other words, create a class that is a `Collection` but is not a `Set` or a `List`) must exercise care if they choose to override the `Object.equals`. It is not necessary to do so, and the simplest course of action is to rely on `Object`'s implementation, but the implementor may wish to implement a "value comparison" in place of the default "reference comparison." (The `List` and `Set` interfaces mandate such value comparisons.)

The general contract for the `Object.equals` method states that equals must be symmetric (in other words, `a.equals(b)` if and only if `b.equals(a)`). The contracts for `List.equals` and `Set.equals` state that lists are only equal to other lists, and sets to other sets. Thus, a custom `equals` method for a collection class that implements neither the `List` nor `Set` interface must return `false` when this collection is compared to any list or set. (By the same logic, it is not possible to write a class that correctly implements both the `Set` and `List` interfaces.)

Specified by:

`equals` in interface `Collection<K>`

Specified by:

`equals` in interface `Set<K>`

Overrides:

`equals` in class `Object`

Parameters:

`o` - object to be compared for equality with this collection

Returns:

`true` if the specified object is equal to this collection

See Also:

`Object.equals(Object)`, `Set.equals(Object)`, `List.equals(Object)`

getMap

```
public ConcurrentHashMap<K,V> getMap()
```

Returns the map backing this view.

Returns:

the map backing this view

clear

```
public final void clear()
```

Removes all of the elements from this view, by removing all the mappings from the map backing this view.

Specified by:

`clear` in interface `Collection`<K>

size

```
public final int size()
```

Description copied from interface: `Collection`

Returns the number of elements in this collection. If this collection contains more than `Integer.MAX_VALUE` elements, returns `Integer.MAX_VALUE`.

Specified by:

`size` in interface `Collection`<K>

Returns:

the number of elements in this collection

isEmpty

```
public final boolean isEmpty()
```

Description copied from interface: `Collection`

Returns `true` if this collection contains no elements.

Specified by:

`isEmpty` in interface `Collection<K>`

Returns:

`true` if this collection contains no elements

toArray

```
public final Object[] toArray()
```

Description copied from interface: `Collection`

Returns an array containing all of the elements in this collection. If this collection makes any guarantees as to what order its elements are returned by its iterator, this method must return the elements in the same order. The returned array's runtime component type is `Object`.

The returned array will be "safe" in that no references to it are maintained by this collection. (In other words, this method must allocate a new array even if this collection is backed by an array). The caller is thus free to modify the returned array.

Specified by:

`toArray` in interface `Collection<K>`

Returns:

an array, whose runtime component type is `Object`, containing all of the elements in this collection

toArray

```
public final <T> T[] toArray(T[] a)
```

Description copied from interface: `Collection`

Returns an array containing all of the elements in this collection; the runtime type of the returned array is that of the specified array. If the collection fits in the specified array, it is returned therein. Otherwise, a new array is allocated with the runtime type of the specified array and the size of this collection.

If this collection fits in the specified array with room to spare (i.e., the array has more elements than this collection), the element in the array immediately following the end of the collection is set to `null`. (This is useful in determining the length of this collection *only* if the caller knows that this collection does not contain any `null` elements.)

If this collection makes any guarantees as to what order its elements are returned by its iterator, this method must return the elements in the same order.

Specified by:

`toArray` in interface `Collection`<K>

Type Parameters:

T - the component type of the array to contain the collection

Parameters:

a - the array into which the elements of this collection are to be stored, if it is big enough; otherwise, a new array of the same runtime type is allocated for this purpose.

Returns:

an array containing all of the elements in this collection

toString

```
public final String toString()
```

Returns a string representation of this collection. The string representation consists of the string representations of the collection's elements in the order they are returned by its iterator, enclosed in square brackets ("[]"). Adjacent elements are separated by the characters ", " (comma and space). Elements are converted to strings as by `String.valueOf(Object)`.

Overrides:

`toString` in class `Object`

Returns:

a string representation of this collection

containsAll

```
public final boolean containsAll(Collection<?> c)
```

Description copied from interface: `Collection`

Returns `true` if this collection contains all of the elements in the specified collection.

Specified by:

`containsAll` in interface `Collection`<K>

Parameters:

c - collection to be checked for containment in this collection

Returns:

`true` if this collection contains all of the elements in the specified collection

See Also:

`Collection.contains(Object)`

removeAll

```
public boolean removeAll(Collection<?> c)
```

Description copied from interface: `Collection`

Removes all of this collection's elements that are also contained in the specified collection (optional operation). After this call returns, this collection will contain no elements in common with the specified collection.

Specified by:

`removeAll` in interface `Collection`<K>

Parameters:

`c` - collection containing elements to be removed from this collection

Returns:

`true` if this collection changed as a result of the call

See Also:

`Collection.remove(Object)`, `Collection.contains(Object)`

retainAll

```
public final boolean retainAll(Collection<?> c)
```

Description copied from interface: `Collection`

Retains only the elements in this collection that are contained in the specified collection (optional operation). In other words, removes from this collection all of its elements that are not contained in the specified collection.

Specified by:

`retainAll` in interface `Collection`<K>

Parameters:

`c` - collection containing elements to be retained in this collection

Returns:

`true` if this collection changed as a result of the call

See Also:

`Collection.remove(Object)`, `Collection.contains(Object)`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.

Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.